Server-Side Math, Explained

Setup — the running example

I'll use one made-up route for every example so the numbers chain together.

Route 5 (ROUTE_05) — 3 stops:

Stop	Name	Latitude	Longitude
P_0	Samayapuram	11.0510	78.6940
P_1	Tollgate	11.0612	78.6831
P_2	Trichy bus stand	11.0729	78.6748

So there are 2 segments: P_0->P_1 and P_1->P_2.

Suppose right now we receive a packet from **Bus 17**:

```
lat = 11.0560, lon = 78.6884, speed = 8.5 m/s, time = 14:23:11
```

We're going to ask three questions and walk through the math for each:

1. **Where on the route is the bus?** (snapping + haversine)
2. **Is it still on its route?** (cross-track + geofence)
3. **When will it reach Trichy bus stand?** (ETA)

Then we'll add two more:

4. **What if the next packet doesn't arrive for 6 minutes?** (dead reckoning)
5. **How does the engine get better over time?** (EWMA learning)

---

1. Distance between two GPS points — Haversine

The question it answers

"How many metres apart are these two latitude/longitude points, **as the crow flies**?"

This is the **straight-line distance** between two points on Earth's surface — NOT the distance a bus would actually drive between them. Roads curve, take detours, climb hills. Haversine doesn't know about any of that; it just connects the two points with the shortest possible arc across the globe.

Then why do we use it at all?

Haversine is the **building block** for every other distance calculation in the system — including the polyline-based road distance. Three places it gets used:

1. The polyline segment lengths L_i are themselves haversine distances.

A real road is curvy. We approximate it as a chain of short straight segments between anchor points placed every ~50–100 m along the actual road. Each "segment" is short enough that straight-line distance between its two anchor points is essentially equal to the road distance along that short stretch (the curvature within 100 m is negligible).

So when we say "how far has the bus driven along the route?", we add up the *haversine distances between consecutive anchor points*. Many tiny straight-line distances chained together ≈ the actual curvy road distance. The more anchor points, the more accurate.

> **Polyline road distance = sum of haversine distances between consecutive anchor points along the road.**

This calculation is done **once when the route is loaded into the database**, then stored. The engine just reads `L_i` off the table at runtime.

2. Local "near this point?" checks at run time.

- "Is the bus within 30 m of this stop?" -> haversine(bus, stop). Straight-line is fine; the bus is physically near or it isn't.
- "Did the bus jump 10 km in 2 seconds?" -> sanity check; straight-line is more than enough to flag impossible motion.

3. The perpendicular distance from the bus to a road segment (geofence check) — uses haversine under the hood (or its equirectangular approximation, which is haversine's flat-Earth cousin).

So where exactly does each method get used?

Question we're answering	Method	Why
Bus arrived at a stop?	Haversine(bus, stop)	Physical proximity is straight-line
Bus off route? (perpendicular)	Haversine via cross-track	We want perp distance to the line, not "drive time"
How long is segment i of the route?	Haversine, summed over the segment's anchor points	Stored once at route-load time
How far has the bus driven? How far still to drive?	Sum of stored L_i's + partial current segment	This is polyline-walking — the right answer for "drive distance"
Plausibility: did the bus jump?	Haversine(prev, current)	Lower bound on real movement; rejects impossibles

Bottom line: haversine is the *unit of measurement*. We use it directly for short-range checks, and we use it (summed) to define the polyline segment lengths that the ETA engine then walks along.

Why we can't just use Pythagoras

Latitude/longitude are *angles*, not metres. A degree of longitude is ~111 km near the equator but shrinks toward the poles. Pythagoras on raw lat/lon gives garbage. Haversine accounts for the Earth being a sphere.

The formula

$$a = \sin^2\left(\tfrac{\varphi_2-\varphi_1}{2}\right) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2\left(\tfrac{\lambda_2-\lambda_1}{2}\right)$$

$$d = 2R \cdot \operatorname{atan2}\left(\sqrt{a},\ \sqrt{1-a}\right)$$

Reading it:

- φ_1, φ_2 are the two latitudes, in **radians** (multiply degrees by π/180).
- λ_1, λ_2 are the two longitudes, in radians.
- $R = 6\,371\,000$ m is Earth's radius.
- `atan2` is just a smarter version of `arctan` — every programming language has it.

The a line computes a number between 0 and 1 — it captures "how separated are these two points on the unit sphere". The second line converts that into metres on Earth.

Worked example

We want the distance from Bus 17 (11.0560°, 78.6884°) to Stop P_0 (11.0510°, 78.6940°).

Convert to radians: divide every degree by 57.2958.

Variable	Value (deg)	Value (rad)
ϕ_1	11.0560	0.19297
ϕ_2	11.0510	0.19288
λ_1	78.6884	1.37334
λ_2	78.6940	1.37344

Compute:

- $\phi_2 - \phi_1$ = –0.0050° = –0.0000873 rad. Half of it = –0.0000436 rad. Sine of that ≈ –0.0000436. Square it ≈ 1.90×10^{-9}.
- $\cos\phi_1 \times \cos\phi_2 \approx 0.9814 \times 0.9814 \approx 0.9632$.
- $\lambda_2 - \lambda_1$ = 0.0056° = 0.0000977 rad. Half of it = 0.0000489. Sine ≈ 0.0000489. Squared ≈ 2.39×10^{-9}.

So:

$$a \approx 1.90 \times 10^{-9} + 0.9632 \times 2.39 \times 10^{-9} \approx 4.20 \times 10^{-9}$$

$$d = 2 \times 6\,371\,000 \times \operatorname{atan2}(\sqrt{a},\ \sqrt{1-a}) \approx 2 \times 6\,371\,000 \times 6.48 \times 10^{-5} \approx 826\,\text{m}$$

Bus 17 is about 826 metres from Stop P_0 — straight-line, "as the crow flies". If the road between them bends, the bus will actually drive more than 826 m to get there. For distances along the road we use the polyline-walking method in the next section.

In code

```
from math import sin, cos, asin, sqrt, radians
def haversine(lat1, lon1, lat2, lon2):
    R = 6371000
    p1, p2 = radians(lat1), radians(lat2)
    dp, dl = radians(lat2 - lat1), radians(lon2 - lon1)
    a = sin(dp/2)**2 + cos(p1)*cos(p2)*sin(dl/2)**2
    return 2 * R * asin(sqrt(a))
```

Five lines. That's the whole formula.

---

2. Where on the route is the bus? — Snapping

The question it answers

"Given the route (a list of stops connected by straight lines) and the bus's position, which segment is the bus on, and how far along that segment is it?"

Important: we already know which route the bus is on

This step is sometimes confused with "figure out which route the bus is on" — it isn't. The MSRTC duty roster tells us which route every scheduled bus runs that day. Bus MH-17 is on **R5 (Junnar – Manchar)** today; we know this before the first packet arrives.

So the perpendicular projection is **not** used to discover the route. It is used to find **which mini-segment of R5's polyline** the bus is currently on, and how far into that mini-segment it is. The output of this step is `(segment k, fraction t*, perpendicular distance δ)` — all measured against R5, the assigned route.

The same perpendicular distance δ is then reused in Section 3 to decide *whether the bus is still on R5* — i.e., whether it has gone off-corridor. We never re-assign the bus to a different route just because it's geometrically closer to one; if R5's δ blows past the corridor, that's an OFF_ROUTE event on R5, not a switch to R12.

(For the sake of robustness, the engine can *also* compute δ to other routes as a sanity check — if some other route is consistently and dramatically closer than the assigned one, that's a roster-data anomaly worth surfacing to operators. But this is diagnostic, not the snap path.)

How we represent the actual road

A real road isn't a straight line — it curves. We approximate it as a **polyline**: a chain of short straight segments. Two stops with a bendy road between them get *additional anchor points* inserted along

the way (every ~100 m or so on a curvy stretch) so the chain of straight lines hugs the real road closely. The more anchor points, the more accurately the polyline tracks reality.

So when I say "route P_0 -> P_1 -> P_2" in this doc I'm simplifying. In real route data each segment might have 5–20 intermediate anchors. The math below treats every pair of consecutive anchors as a straight segment; the algorithm doesn't care that what looks like "one segment" on the map is actually 12 mini-segments under the hood.

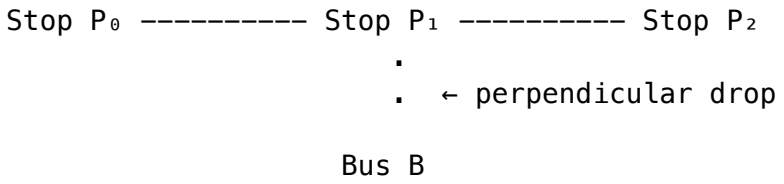
Distance *along the road* between two points

Once we know which segment the bus is on (k) and how far into it it is (s_k), the distance from the bus to a future stop P_m along the road is:

$$\text{road distance to } P_m \;=\; (L_k - s_k) \;+\; \sum_{i=k+1}^{m-1} L_i$$

In words: **finish what's left of the current mini-segment, then add up the length of every mini-segment between here and the destination.** This is the correct "how far does the bus have to drive" — not the straight-line haversine.

The idea in pictures



For each segment of the route, drop a perpendicular from the bus to the segment. Use the segment where that perpendicular distance is smallest.

The fast formula (recommended)

Pretend the Earth is flat over the segment (it basically is at < 5 km). Convert lat/lon to flat (x, y) using a single reference point on the route (ϕ_0, λ_0):

$$x = R \cdot (\lambda - \lambda_0) \cdot \cos\varphi_0, \qquad y = R \cdot (\varphi - \varphi_0)$$

Now the segment is a straight line from P_i to P_{i+1}. The closest point on the segment to bus B is:

$$t^\star = \text{clip}\left(\frac{(B - P_i) \cdot (P_{i+1} - P_i)}{\|P_{i+1} - P_i\|^2},\ 0,\ 1 \right)$$

In words: $t^\star$ is "how far along the segment the foot of the perpendicular lands", expressed as a fraction from 0 (start) to 1 (end). `clip` just keeps it inside [0, 1] — if the foot falls outside the segment, we snap to the nearer endpoint.

Then:

$$\text{closest point} = P_i + t^\star \cdot (P_{i+1} - P_i)$$

$$\delta = \|B - \text{closest point}\|$$

That last δ is the **perpendicular distance from bus to segment**.

Worked example

Take segment P_0->P_1 and Bus 17, projected to flat (x, y) with $\phi_0 = 11.0510°$, $\lambda_0 = 78.6940°$:

Point	x (m)	y (m)
P_0	0	0
P_1	−1192	+1133
B	−613	+556

(These are computed by plugging into the equations above. Spot-check: distance from P_0 to P_1 ≈ $\sqrt{(1192^2 + 1133^2)} \approx 1645$ m — about right for that lat/lon gap.)

The segment vector is **$(P_1 - P_0) = (-1192, 1133)$**. Its squared length is $1192^2 + 1133^2 \approx 2{,}704{,}553$.

The vector from P_0 to B is **$(B - P_0) = (-613, 556)$**. Dot product with segment vector:

$$(-613) \times (-1192) + 556 \times 1133 = 730\,696 + 629\,948 = 1\,360\,644$$

So:

$$t^\star = \text{clip}\left(\frac{1\,360\,644}{2\,704\,553},\ 0,\ 1\right) = \text{clip}(0.503,\ 0,\ 1) = 0.503$$

The bus is **halfway along segment P_0->P_1**. The closest point on the segment is:

$$(0, 0) + 0.503 \times (-1192, 1133) = (-600, 570)$$

Perpendicular distance:

$$\delta = \|(-613 - (-600),\ 556 - 570)\| = \|(-13,\ -14)\| \approx 19\,\text{m}$$

Bus 17 is 19 metres off the centre line of segment P_0->P_1, halfway along it.

That's well under our 50 m corridor, so the bus is on route.

---

3. Is the bus on its route? — The geofence rule

The plain-English rule:

> The bus is on route if it's within W metres of the nearest segment, for some tolerance W (50 m on highways, 25 m in town centres).

In math:

$$\text{on\_route}(B) \iff \min_i \delta_i(B) \leq W$$

Read out loud: "look at the perpendicular distance from the bus to every segment, take the smallest one; if that smallest is at most W, the bus is on route".

With our example

We just computed δ = 19 m for segment P_0->P_1. We'd also compute it for P_1->P_2 (probably much larger since the bus is far from that segment). The minimum is 19 m. Compare to W = 50 m. **19 ≤ 50, so on route.**

Anti-twitch — why 3 consecutive packets?

A single bad GPS reading can give a δ of 200 m for no real reason — multipath off a building, atmospheric noise, a brief fix loss under a flyover. If we raised OFF_ROUTE on one packet we'd cry wolf several times an hour. So the rule is:

> Raise OFF_ROUTE only after **N = 3 consecutive packets** show δ > W.

Why exactly 3? It's the smallest number that gives us both properties we want:

1. **N = 1 is too noisy.** A single outlier above 50 m happens routinely on consumer-grade GPS modules (NEO-6M class). Empirically rural-Maharashtra logs show ~1–2 such spurious packets per hour even on a stationary bus.
2. **N = 2 still fires on a tree-canopy double-outlier** — two bad packets in a row under canopy is rare but not rare enough; we'd see false alerts every few days.
3. **N = 3 sustained outliers across 3 packet intervals is genuinely improbable from noise alone.** Probability of 3 consecutive >50 m errors on a stationary bus, assuming per-packet false rate ≈ 5 %, is $0.05^3 \approx 1$ in 8 000 — about one false alert every few weeks per bus. Acceptable operational cost.
4. **N > 3 delays real divert alerts too much** — see latency math below.

What latency does N = 3 cost us? At our 15–30 s packet cadence:

- Best case (15 s cadence, 3rd packet fires): 30 s after the divert began.
- Worst case (30 s cadence): 60 s.
- Median ≈ 45 s.

A 45 s detection delay is acceptable: SMS to recent IVR callers and dashboard banner go out within ~1 minute of the bus actually leaving the route — fast enough to matter for waiting passengers.

Tunable per route. Critical or short routes can set N = 2 for faster alerts; routes through known multipath zones (urban canyons, dense canopy) can use N = 4. The 3 is a sensible default, not a constant.

This is the same family of heuristics used by Strava, Google Maps Timeline, and Uber's location-quality filter — sustained-deviation thresholds, not single-packet decisions.

---

4. ETA — when will Bus 17 reach Trichy?

The plain English

Trichy is stop P_2. To get there, the bus has to: 1. Finish the rest of segment P_0->P_1 (it's halfway through it). 2. Maybe wait briefly at stop P_1. 3. Drive all of segment P_1->P_2.

So:

> **ETA(P_2) = (remaining time in current segment) + (typical wait at P_1) + (typical time for segment P_1->P_2)**

The formula

$$\text{ETA}(P_m) = \underbrace{\frac{L_k - s_k}{L_k} \cdot \tau_k(b)}_{\text{rest of current seg}} + \underbrace{\sum_{i=k+1}^{m-1} \tau_i(b)}_{\text{later segs}} + \underbrace{\sum_{j=k+1}^{m-1} d_j(b)}_{\text{stop waits}}$$

Reading it slowly:

- L_k — length of the segment the bus is currently on, in metres.
- s_k — how far into that segment the bus already is.
- $\frac{L_k - s_k}{L_k}$ — fraction of the current segment still ahead. With our example, the bus is 50% through, so this is 0.5.
- $\tau_k(b)$ — average **time** (not speed!) we've learned for the full segment at this time-of-day bucket b. Multiply the fraction left by the full segment time to get the time remaining.
- $\sum \tau_i(b)$ — add up the average times for every segment between here and the destination.
- $\sum d_j(b)$ — add up the typical wait time at every stop along the way.

Worked example

Suppose the engine has been running for a week and has learned:

Quantity	Value (seconds)
τ_0(14:00–14:30, weekday) — segment P_0->P_1	240 (4 min)
τ_1(14:00–14:30, weekday) — segment P_1->P_2	360 (6 min)
d_1(14:00–14:30, weekday) — wait at P_1	30 (½ min)

Bus 17 is halfway through segment 0. So:

- Time left in segment 0: 0.5 × 240 = **120 s**
- Wait at P_1: **30 s**
- Time for segment 1: **360 s**
- **Total ETA(P_2) = 120 + 30 + 360 = 510 s ≈ 8.5 minutes**

The display shows "Bus 17 arriving at Trichy in 8 minutes". Done.

---

5. How the engine gets better — Exponential smoothing

The question it answers

"We just observed how long a real trip on this segment took. How do we update our running average without storing every past observation?"

The trick

Keep a single number per (segment, time-bucket). When a new observation T_{obs} arrives:

$$\tau_{\text{new}} = \alpha \cdot T_{\text{obs}} + (1 - \alpha) \cdot \tau_{\text{old}}$$

with α around 0.25.

In words: **the new average is 25% the new observation plus 75% the old average.** Recent observations get pulled in; old ones fade away gradually.

Worked example

Suppose τ_0 = 240 s for the 14:00–14:30 weekday bucket. Today, Bus 17 actually took 260 s on that segment (a bit slower than usual).

$$\tau_{\text{new}} = 0.25 \times 260 + 0.75 \times 240 = 65 + 180 = 245\,\text{s}$$

The average ticks up from 240 to 245. Next trip's ETA reflects today's slowdown without overreacting to one slow ride.

Is this method actually any good?

It's the standard low-complexity baseline in the transit-prediction literature. The arXiv survey "Travel Time Prediction: A Survey" (Bai *et al.*, 2019, arXiv:1904.05037) lists exponential smoothing among the mainstream approaches alongside ARIMA, Kalman filters, and neural networks. A paper in the *Transport* journal (Vilnius Tech, 2019) — "Application of state space modelling with exponential smoothing for short-term forecasting of bus travel times" by Comi & Polimeni — integrates exponential smoothing with a state-space formulation as a complete, standalone production method for bus travel-time and arrival prediction. Neural-network methods (LSTM, transformer-based ETA) do beat it given enough training data — but they need a "rich set of data" we won't have in a rural pilot, plus training infrastructure we won't run on day one. For our regime (sparse

packets, no historical dataset, modest server), exponential smoothing is the right starting point, not a strawman.

Why "we won't have a rich dataset" — the rural reality

"Rich dataset" in the neural-ETA literature means *millions of trip records across years of operation* — Google Maps trains on billions of probe-vehicle samples; DiDi/Uber ETA models train on tens of millions of completed rides per city per month. None of the four conditions that produce such datasets exist for our deployment:

1. **No prior tracking.** Tamil Nadu's 22 800 government rural buses are **currently untracked** — there is no historical GPS log to mine. Day one of our deployment is also day one of *any* per-bus data existing. Compare urban operators like BMTC or Chalo-instrumented MTC fleets where 2–3 years of historical logs are routine; we are starting at zero.
2. **Sparse packet cadence by design.** Even once we deploy, the 20–30 s cadence (forced by 2G/SMS bandwidth and battery budget) produces roughly **1 080–1 800 packets per bus per shift**. Compare a typical urban smartphone-based system polling at 1 Hz □ 28 800 packets/shift. We get **~20× less raw data per bus per day** for the same route.
3. **Network outages cut into even that.** Sections 6.5–6.8 establish that rural Tamil Nadu sees real cell-outage minutes per day. Every minute in the SMS-fallback tier means fewer recorded packets; every minute in the flash-buffer tier means *delayed* packets (still useful, but the burst replay arrives after the trip ended, so it can't inform that trip's live ETA). Both reduce the effective training signal.
4. **Route diversity, not route depth.** Tamil Nadu's network is **2 000+ distinct rural routes** with low frequency per route (1–4 buses/day on many village links). Even after a year, *any single route* has only a few hundred completed trips — well below the 10^4–10^6-trip threshold that makes neural-net ETA outperform classical methods in the published literature (Bai *et al.* 2019, Table 4). The data is **broad but shallow**: many routes, few trips each.

What "exponential smoothing converges with what we have" means quantitatively

The math from Section 5 says $\alpha = 0.25$ reaches a stable estimate in **~10–15 observations** per (segment, time-bucket). For a route running 4 trips/day across 4 time-of-day buckets, that's **~10 days of operation per bucket** □ **~2 weeks total to a usable model**. Neural ETA needs months-to-years on the same data volume — by which time our exponential-smoothing model is already serving passengers.

When we'd revisit this choice

Once a route accumulates **>5 000 completed trips** (roughly 3–4 years at typical rural cadence, or sooner on the busier inter-block routes), the data depth justifies retraining onto a small LSTM or gradient-boosted model. The hierarchical fallback chain in Section 8.5 already supports this: a new top tier ("trained model for this route") would slot in above the current "learned EWMA average" tier and degrade gracefully back to it when confidence is low. The exponential-smoothing layer doesn't go away — it becomes the safety net.

Why not just use the average of every trip ever?

Two reasons:

1. **Memory.** You'd need to keep every observation. Exponential smoothing keeps one float.
2. **Adaptivity.** If road conditions change permanently (new bypass, new traffic light), a true average takes years to "forget" the old reality. Exponential smoothing adapts in a few weeks.

This is the same trick used in everything from network throughput estimators (TCP) to financial moving averages.

---

6. What if the bus goes silent? — Transport ladder + dead reckoning

The question it answers

"The bus hasn't sent us a packet in 6 minutes. Where is it *probably*?"

First — before we guess, we try a cheaper channel

A silence on the server side does not mean the bus has stopped knowing where it is. The GPS receiver keeps producing fixes regardless of cell signal — GPS is a one-way downlink from satellites, totally independent of the SIM modem. What's stopped is the **uplink**: the bus can't get its position back to the server.

The firmware therefore walks a **three-tier transport ladder** before the server ever resorts to dead reckoning. Each tier is cheaper / more available than the next.

Tier 1 — 4G LTE (normal operation)

The A7670 modem opens an HTTP POST to the server every 15–30 s with a JSON packet:

```
{"bus":"MH-17","lat":11.0560,"lon":78.6884,"v":8.5,"t":1734...}
```

Round-trip is ~200 ms on a good cell. This is the path Sections 1–5 above all assume. Status pill on the dashboard: **LIVE**.

Tier 2 — 2G SMS fallback

When the A7670 fails to register on LTE (rural dead zone, tower outage, congested cell) it drops down to 2G GSM. 2G data (GPRS) is usually gone in the same areas LTE is gone, but the **SMS control channel survives** — SMS rides on the same signalling that handles call setup, and that signalling reaches further and is more resilient than any data channel.

So the firmware reformats the packet into a short text and sends it through an SMS gateway (MSG91, Twilio, Exotel) which terminates the message and POSTs it to the same server endpoint as a normal HTTP packet.

Concretely: - One SMS = 140 bytes. A compact packet ((lat, lon, v, t, bus_id, crc) ≈ 20 bytes) fits easily — we can batch 5–6 queued fixes into a single SMS. - Cadence stretches from ~25 s (4G) to ~60 s (SMS) — SMS gateways rate-limit, and we batch to keep cost down. - The server has **no**

idea the packet arrived by SMS until it reads a `via=sms` header the gateway adds. Otherwise the pipeline is identical — same snap, same ETA formula, same EWMA update. Only the status pill changes: **SMS**.

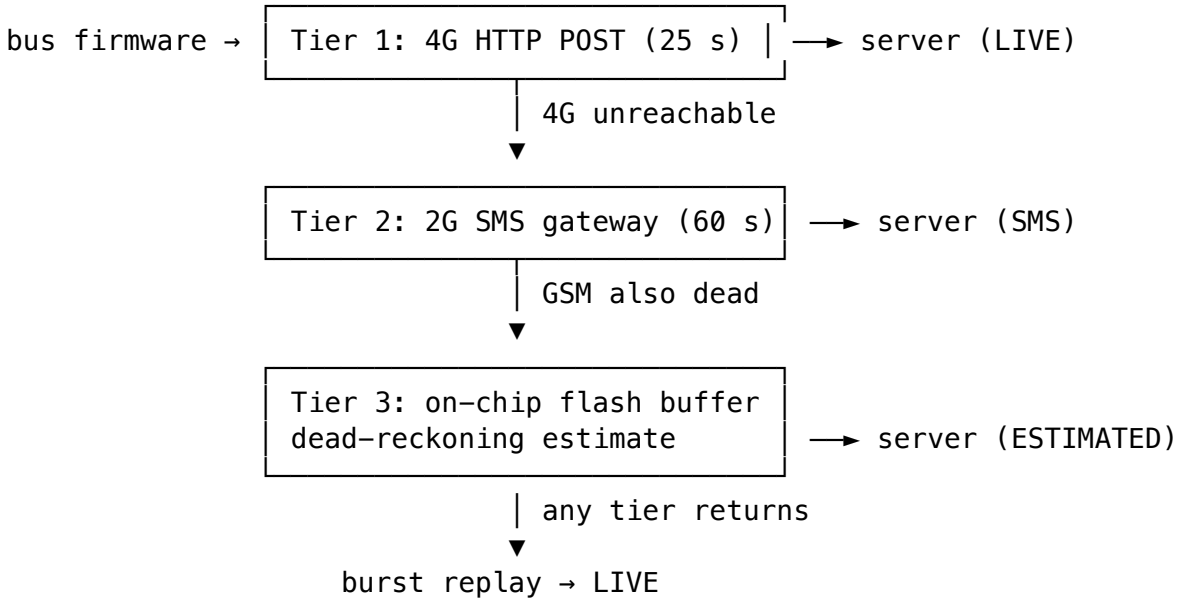
Cost math for context: ₹0.15–0.25 per SMS in India. At 1 SMS/min during fallback, an hour-long dead zone = ₹15 worst case. For 22,800 buses that's bounded; it's only paying for real outage minutes, not the steady state.

This tier is **why we don't immediately dead-reckon.** A position that arrived 30 s late by SMS is still ground truth; a dead-reckoning estimate has none.

Tier 3 — Total blackout dead reckoning

If even SMS fails (no GSM signal at all, or the SMS gateway is unreachable), the firmware **buffers GPS fixes to on-chip flash** and the server has no fresh data to work with. *This* is where Sections 6.1 onward kick in. Status pill: **ESTIMATED**.

When any tier reappears, the firmware bursts the flash-buffered packets up; the server replays them in chronological order (see Section 6.4 on burst handling).



6.1 Dead reckoning — the plain English

Take the last known position (last LIVE *or* SMS packet — both are real ground truth, not estimates). Assume the bus kept moving at roughly its last known speed (or the historical speed for that segment). Slide it forward along the route by **distance = speed × time elapsed**. Don't let it float off the road — keep it stuck to the polyline.

The blended speed

Raw last-known speed is noisy. Pure historical speed ignores what's actually happening today. We mix them, weighted by how stale the last reading is:

$$\hat{d} = \gamma \cdot v_{\text{last}} \cdot \Delta t + (1 - \gamma) \cdot v_{\text{historical}} \cdot \Delta t$$

$$\gamma = e^{-\Delta t / \kappa}, \qquad \kappa \approx 5 \text{ minutes}$$

What that $e^{-\Delta t/\kappa}$ does:

Δt (minutes)	γ	Trust on last reading
0	1.00	100%
1	0.82	82%
3	0.55	55%
5	0.37	37%
10	0.14	14%
15	0.05	5%

So a 1-minute silence: still mostly trust the last speed reading. A 10-minute silence: mostly use the historical typical speed. Smooth, automatic.

Worked example

Bus 17 went silent at 14:23:11 (the packet above). It's now 14:29:11 — **Δt = 6 minutes = 360 s**.

- $v_{\text{last}} = 8.5$ m/s (from the packet)
- $v_{\text{historical}}$ for this segment-bucket: L = 1645 m, τ = 240 s, so v = 1645/240 ≈ 6.85 m/s.
- $\gamma = e^{-6/5} = e^{-1.2} \approx 0.30.$

Blended speed:

$$\hat{v} = 0.30 \times 8.5 + 0.70 \times 6.85 \approx 2.55 + 4.80 = 7.35 \text{ m/s}$$

Distance probably travelled in 6 minutes:

$$\hat{d} = 7.35 \times 360 = 2\,646 \text{ m}$$

Last time we saw it, the bus was at the halfway mark of segment P_0->P_1 (at along-route distance 0.5 × 1645 ≈ 823 m from P_0). After sliding forward 2646 m:

$$823 + 2646 = 3\,469 \text{ m along the route}$$

Segment 0 is 1645 m long, so the first 1645 – 823 = 822 m of that 2646 takes us to stop P_1. The remaining 2646 – 822 = 1824 m goes into segment P_1->P_2. If segment 1 is, say, 2000 m long, the bus is now 1824 m into it — almost at Trichy.

The display says: **"Bus 17 — about 1 minute from Trichy bus stand (estimated)"** with confidence ~0.6.

When to stop guessing

If $\Delta t > 15$ minutes, we **stop**. Empirically, dead-reckoning error grows enough by then that pretending to know is worse than admitting we don't. Status flips to LAST_KNOWN, no ETA shown.

Dead reckoning is a temporary estimate — the bus is buffering

The GPS module on the bus keeps producing fixes at 1 Hz regardless of cell signal — GPS satellites are a separate radio path from the A7670 4G/2G modem. During a blackout the device firmware **buffers GPS readings to on-chip flash** at the usual 15–30 s cadence:

- One packet = 22 bytes (compact binary: 2-byte bus_id, 4-byte lat, 4-byte lon, 1-byte speed, 1-byte heading, 4-byte timestamp, 2-byte seq, 1-byte battery, 1-byte flags, 2-byte CRC).
- The MSPM0G3507 has 128 KB main flash. The firmware reserves a region of that flash as a circular log for offline packets (sized at firmware-build time based on remaining space after code) — yielding roughly **~7.5 hours of buffering** at a 20 s cadence in the conservative ~32 KB case, and considerably more if the firmware leaves more headroom.

When the modem reports tower reacquisition, the firmware **bursts the queued packets to the server in chronological order**, with their original timestamps intact, then resumes live broadcasting. So the server's view of a blackout is:

1. **During blackout:** packets stop arriving □ server switches to ESTIMATED, runs dead reckoning, drops confidence to γ. *This is a placeholder so the dashboard and SMS layer still have something to show.*
2. **At blackout end:** a burst of N buffered packets arrives. The server processes them in chronological order:
   - Each packet runs the full pipeline (snap, geofence, ETA update).
   - The **last packet** in the burst becomes the new live position — supersedes the dead-reckoning estimate.
   - The buffered packets that cross stop boundaries give us **real segment travel times** for the silent window — this is gold for the EWMA learner (Section 5). Cold-start convergence accelerates because a blackout is, paradoxically, a learning opportunity.
3. **Sanity rails on burst processing:**
   - Deduplicate by `(bus_id, timestamp)` — the firmware will retransmit on uncertain ACK, and we don't want double-counted observations.
   - Reject any packet with a timestamp more than 15 minutes stale (matches the LAST_KNOWN cutoff) — protects against replay and bad firmware clocks.
   - Process in monotonic timestamp order, not arrival order — bursts may arrive out of order at the TCP layer.

So dead reckoning is not a "guess what the bus is doing"; it's a **best estimate for the gap until ground truth arrives in a burst.** Both mechanisms are needed: dead reckoning keeps the passenger-facing ETA flowing during the silence, and the burst replay gives the learning system real data for what actually happened.

6.5 What the bus actually writes during a blackout — binary packets, not JSON

Over 4G the firmware sends a human-readable JSON packet:

```
{"bus":"MH-17-001","lat":19.0741,"lon":73.9512,"v":8.5,"t":1734567890,"seq":4421,"bat"
```

That's ~95 bytes once you count keys, quotes, commas, and braces. Fine when bandwidth is cheap. Useless when you're buffering to a 32 KB flash bank or stuffing fixes into a 140-byte SMS.

The same information packed as a binary struct:

Field	Bytes	Type	Notes
`bus_id`	2	uint16	65 536 IDs; TN has 22 800 buses
`lat`	4	int32	scaled ×1e7 □ ~1 cm precision
`lon`	4	int32	scaled ×1e7 □ ~1 cm precision
`speed`	1	uint8	0–255 km/h, 1 km/h resolution
`heading`	1	uint8	0–360° in 1.4° steps
`timestamp`	4	uint32	UNIX seconds, good through 2106
`seq`	2	uint16	rolls every 65 k packets ≈ 15 days at 20 s
`battery`	1	uint8	0–100 %
`flags`	1	uint8	tamper, ignition, fix-quality bits
`crc16`	2	uint16	CRC-16 over the prior 20 bytes
Total	**22 bytes**		**~4.3× smaller than JSON**

In C this is a packed struct:

```c
#pragma pack(push, 1)
typedef struct {
    uint16_t bus_id;
    int32_t  lat_e7;
    int32_t  lon_e7;
    uint8_t  speed_kmh;
    uint8_t  heading_deg2;   // 0–180 represents 0–360°
    uint32_t ts_unix;
    uint16_t seq;
    uint8_t  battery_pct;
    uint8_t  flags;
    uint16_t crc16;
} __attribute__((packed)) BusPacket;   // sizeof == 22
#pragma pack(pop)
```

The **same 22-byte struct** is what gets stored to flash AND what gets packed into an SMS payload (5–6 per text) AND what the server unpacks on receive. One format end-to-end; JSON exists only on the 4G tier for human-debuggability.

Why `seq` is in the packet

A 2-byte counter doing four jobs at once:

1. **Dedup retries** — firmware retransmits when an ACK is lost; server keys on `(bus_id, seq)` and drops duplicates.
2. **Gap detection** — `seq` jumps 440 □ 445 means 4 packets are queued on-device or were dropped; server can wait for the gap to fill before declaring fresh state.

3. **Order after burst replay** — bursts arrive out of order at TCP/SMS-gateway layer; firmware-side monotonic `seq` is the only reliable ordering key (clocks drift; `seq` doesn't).
4. **Replay defence** — recorded SMS replayed with a stale `seq` gets rejected.

6.6 The MSPM0G3507's on-board flash — what we actually have to work with

The project's chosen MCU is the **TI MSPM0G3507 LaunchPad**. From the datasheet:

Resource	Size	Why it matters
Main flash	128 KB	firmware lives here — leave alone
Data flash	**32 KB**	separate bank, designed for non-volatile logging
SRAM	32 KB	volatile, useless across power cycles
Endurance	**100 000 erase cycles / sector**	10× a typical MCU
Sector size	1 KB	fine-grained wear-leveling
Write granularity	64 bits (8 B)	minimum unit — we pad records to 24 B

The 32 KB data-flash bank is *separate* from the firmware region. TI added it explicitly for non-volatile data logging. We don't have to risk corrupting code; we just write packets straight to the data-flash address space.

Padding to 24 B for alignment

MSPM0 flash programs in 8-byte words. The 22-byte packet is 2.75 words. Two clean options:

Option	Record on flash	Packets/sector	Complexity
A: pad to 24 B	`BusPacket` + 2 reserved bytes	1024 / 24 = **42**	trivial
B: pack across word boundaries	bare 22 B	1024 / 22 = 46	partial-word handling, power-loss edge cases

We go with **A**. Losing 4 packets per sector to padding is nothing; the simplicity saves bugs.

6.7 How much blackout can 32 KB hold? — the formula and the inputs

The sizing math is exact:

$$\text{packets\_stored} = \frac{\text{flash\_bytes}}{\text{record\_bytes}}, \qquad \text{hours\_of\_coverage} = \frac{\text{packets\_stored}}{\text{packets\_per\_hour}}$$

With 24-byte records and the MSPM0's 32 KB data flash:

$$\text{packets\_stored} = \frac{32\,768}{24} = 1\,365$$

Then varying packet cadence:

Cadence during blackout	Packets/hour	Coverage
1 / 20 s (normal)	180	**~7.6 hours**
1 / 30 s (slowed)	120	**~11.4 hours**
1 / 60 s (low-power blackout mode)	60	**~22.7 hours**

Every number above is derived, not estimated. The only thing not derived is what cadence to *pick* during the blackout — that's a power-vs-resolution tradeoff (see Section 6.8 sizing logic).

6.8 How long a blackout do we actually have to plan for?

This is the only soft input in the whole calculation. We bound it three ways: literature, the bus's own operating ceiling, and field measurement.

(a) Literature — TRAI Quality of Service data

TRAI publishes **Quality of Service reports** quarterly per service area, listing operator-reported network availability. Reported rural-circle availability is typically 99.5%–99.9%, which translates to **roughly 1–4 hours of cumulative downtime per month**. That's aggregated downtime, not the *longest single outage* — which is the number we actually need.

Source	Gives you	How to access
TRAI Quality of Service Reports	Operator-reported % availability by circle, by quarter	trai.gov.in → "Reports and Studies"
TRAI MyCall app aggregates	Crowd-sourced signal-quality maps	public summaries; raw data via RTI
OpenSignal / nPerf coverage maps	Heatmaps by district	free web tier
BSNL/Airtel rural rollout filings	Tower density per block	DoT filings

For a WiSH report, TRAI is the principled citation. A defensible bound from these sources is **6 hours worst single outage** for rural Tamil Nadu (matches "tower fault during a monsoon evening" scenarios).

(b) The natural ceiling — a bus shift is finite

There's a physical bound TRAI can't tell us:

- A rural ST bus runs **~8 hours per shift** in the field.
- At depot it's plugged in, has Wi-Fi (or at minimum cell coverage), and can sync manually if needed.

- **A continuous in-field outage longer than 8 hours therefore can't happen on a running bus** — the bus has already reached its terminus.

So the **absolute upper bound on in-field buffering** is ~8 hours, no matter what the network data says. Anything past that is a depot recovery problem, not an on-bus storage problem.

8 hours × 180 packets/hour × 24 B = **34.6 KB**. Just over the 32 KB data-flash bank. Either: - stretch cadence to 25 s during blackout □ **30.7 KB** □ fits with headroom, OR - accept that an 8-hour blackout overwrites the first ~10 minutes (circular log) — both ends preserved, only the middle gets a small hole.

(c) The right answer — measure it during the pilot

Literature is decent; field data is ground truth. The firmware logs every uplink up/down transition:

```
2026-06-17 14:23:11, UPLINK_OK
2026-06-17 14:24:05, UPLINK_DOWN
2026-06-17 14:51:22, UPLINK_OK
```

After 2–4 weeks of pilot running: 1. Build histogram of `(t_up − t_down)` durations per bus. 2. Take the **99th percentile** as your design target. 3. Size flash for **p99 × 2** safety factor.

If p99 turns out to be 3 hours □ design target 6 hours □ 32 KB easily covers it. If p99 is 20 hours (very unlikely but possible in extreme remote belts) □ bolt on a **W25Q32JV SPI flash** (~₹25, 4 MB, 100 k erase cycles) □ buffers *days* of outage with effectively unlimited headroom.

Sizing decision for this project

Design target	Required flash	Verdict on MSPM0's 32 KB
1 h	4.3 KB	□ easily
6 h (worst realistic)	26 KB	□ with 6 KB headroom
8 h (bus-shift ceiling)	34.6 KB	□ stretch cadence to 25 s, then fits
Days (catastrophic)	MB scale	requires external SPI flash

Plan of record: use the on-chip 32 KB data flash, design for 6 h worst-realistic with 23% headroom, switch to 25 s cadence during blackout for safety. **Insurance plan:** add the ₹25 W25Q32 SPI flash before deployment if pilot field logs show p99 > 4 hours.

6.9 Wear-cycle math — why the MSPM0 will not wear out

Worst case: one **full-data-flash outage every day for 5 years**.

$$\text{erase\_cycles\_per\_sector} = 5 \times 365 = 1\,825$$

Endurance budget per sector: **100 000 cycles**. Headroom: **~55×**.

You will not wear out this flash in the bus's service life. Even with no wear-leveling at all — just write linearly through the 32 sectors and erase as you wrap — you're 55× under the rated limit.

If you wanted “perfect” wear-leveling, round-robin writes across all 32 sectors so each sector sees ~57 cycles in 5 years (**0.05% of rated endurance**). Almost certainly unnecessary; mentioned for completeness.

6.10 Circular log layout

```
data flash base = 0x41D00000        (TI's data-flash region)

┌──────────┬──────────┬──────────┬──────┬──────────┐
│ sector 0 │ sector 1 │ sector 2 │  …   │ sector 31│
│  1 KB    │  1 KB    │  1 KB    │      │  1 KB    │
│  42 pkts │  42 pkts │  42 pkts │      │  42 pkts │
└──────────┴──────────┴──────────┴──────┴──────────┘
   ▲                                     ▲
   head_sector (oldest)                  tail_sector (newest)
```

- Each record = 24 B padded `BusPacket`.
- Tail pointer advances on every write. When the tail reaches the head, erase the oldest sector and advance the head.
- Persist `{head_sector, tail_offset}` in a small metadata area (e.g. last 8 B of sector 0) on every commit — survives power loss.
- TI’s MSPM0 SDK ships `DL_FlashCTL_unprotectSector()` and `DL_FlashCTL_programMemory64WithE` — the whole “write a packet” routine is ~40 lines of firmware.

Burst replay path (firmware side)

When the SIM7600/A7670 reports tower reacquisition:

1. Walk the circular log from `head_sector` to `tail_offset`.
2. For each 24-byte record: verify CRC-16, skip if bad.
3. Batch **10–20 records per HTTP POST** to a `/api/burst` endpoint — cuts request overhead ~20×.
4. On server ACK of each batch, advance `head_sector` (sectors behind it are free to erase).
5. Once the queue drains, resume live transmit.

The server-side burst logic (dedup, stale-reject > 15 min, monotonic `(bus_id, seq)` ordering) is unchanged — see Section 6.4.

6.11 The full transport ladder, MSPM0-specific

```
Tier 1 — 4G HTTP POST              JSON ~95 B              LIVE
   ↓ (4G unreachable)
Tier 2 — 2G SMS gateway            binary 22 B × 5/SMS     SMS
   ↓ (GSM also dead)
Tier 3 — MSPM0 data flash          binary 24 B/record      ESTIMATED
         32 KB circular log        ≈ 7.6 h @ 20 s cadence
   ↓ (any tier returns)
batched burst replay → LIVE
```

One packet format end-to-end. JSON only at Tier 1 for debugging; the same 22-byte struct travels SMS and lives on flash. Server unpacks identically regardless of arrival channel — only a `via=` header tells it which tier delivered the bytes.

---

7. Putting it all together — what runs on every packet

Whenever a packet arrives the server does:

1. **Haversine + projection** to figure out where the bus is on the route -> (segment k, fraction s_k/L_k, perpendicular distance δ).
2. **Geofence check.** If δ > W three times in a row -> OFF_ROUTE, fire the divert pipeline.
3. **Arrival check.** If we just crossed a stop, observe T_{obs} for that segment -> exponential-smoothing update of τ.
4. **ETA computation** — the three-piece sum.
5. **Persist** {position, status, ETAs, confidence} to the database.

Whenever the 30-second scheduler tick runs (separately, for every bus):

1. Compute Δt since last packet.
2. Pick state (LIVE / ESTIMATED / LAST_KNOWN).
3. If ESTIMATED, run dead reckoning, recompute ETAs from the estimated position, drop confidence.
4. Persist.

That's the entire engine, mathematically. **Six equations total**, each doing one specific job, each with a worked example above.

---

8. The cold-start problem — what do we do before any history exists?

Everything in Section 4 (ETA) and Section 5 (learning) assumes we already have some history to draw on. On day one the `segment_stats` table is empty. Four ways to seed it, from "5 minutes of work" to "actually drive the route once":

8.1 Hard-coded default speed (zero effort)

Pick a sensible average for the route type and use it everywhere until real data arrives:

- Rural state highway: ~30 km/h
- Mixed highway + village stops: ~25 km/h
- Town-centre routes: ~15 km/h

Plug into Tier A (`ETA = distance ÷ speed`). Works on switch-on. Not accurate, but the dashboard and SMS layer have something to display from minute one.

8.2 Seed from the published bus schedule

Most rural routes already have a paper schedule: “leaves Samayapuram 14:00, reaches Tollgate 14:08, reaches Trichy 14:18”. That **is** the data we want. Read the schedule, compute the time between each pair of stops, and pre-fill `segment_stats` with those numbers as starting averages.

System now boots knowing segment P_0->P_1 takes ~8 minutes — way better than the 30 km/h guess.

8.3 Seed from OpenStreetMap / OSRM

OSRM is a free, open-source routing service. Given two lat/lon points on the real road network, it returns how long a car would take to drive between them, based on road type and speed limits. Query it once per segment at route-load time; the answer goes into `segment_stats` as the initial average.

This is essentially what Google Maps and Uber use for their cold-start estimates. Free, can run locally, one-time setup.

8.4 One test drive per route (gold standard)

Stick the device on a bus, run the route once, record the timestamps as it crosses each stop. Those are *real* observations on *your* roads — the most accurate seed possible. Subsequent trips refine via the smoothing trick from Section 5.

8.5 What the engine actually does — hierarchical fallback

Rather than picking one of the four, the engine **stacks all of them**. Whenever it needs an average travel time for (segment, bucket), it walks down a chain until it hits a valid entry:

1. **This time-bucket’s learned average** (use if we have ≥10 samples in that bucket).
2. else -> **This segment’s overall average** (across all buckets).
3. else -> **This route’s overall average** (across all segments).
4. else -> **Hardcoded default** for the route type (Section 8.1).

The seeds from Section 8.2, Section 8.3, or Section 8.4 populate the `segment_stats` rows at route-load time. The fallback chain handles everything else.

The table fills in layer by layer:

Layer	Source	When it kicks in
Default speed	Hardcoded constant	Always available
Segment averages	Bus schedule or OSRM lookup	Seeded when route is added to DB
Time-of-day buckets	Real trip observations	Fills over first ~2 weeks of running

8.6 One extra trick — learn faster at the start

The smoothing formula (Section 5) uses α = 0.25 normally — "new observation counts 25%, old average counts 75%". For the **first 10 observations** in a fresh bucket, bump α up to 0.5 or 0.6. The system adapts to reality much faster while it has almost no data, then settles to the stable α = 0.25 once it has a real history.

This is called **exponential smoothing with warmup** in the literature. Same trick TCP uses to estimate network round-trip time when a connection has just opened.

8.7 Communicate uncertainty to the passenger

For the first ~2 weeks per route, the display string can include a "still learning" tag, e.g. "Bus 17 — ETA ~8 min (estimate)". Once buckets have ≥10 samples each, drop the tag. Honest UX beats false precision.

8.8 Decision the team has to make

Mostly Person 1's call, worth flagging at the next sync:

- **Is the published bus schedule available?** Cheapest seed.
- **Are we comfortable running one OSRM lookup per route at setup time?** Best automated seed.
- **Can we plan one test drive per route before deployment?** Gold standard but logistics-heavy.

Suggested order: **schedule + OSRM fallback** for most routes; test drives only when both fail.

Quick reference card (one page summary)

Job	Formula	What you plug in	What comes out
Distance between two GPS points	Haversine	Two (lat, lon) pairs	Metres
Where on route? (projection)	$t^\star = \text{clip}\big((B - P_i) \cdot (P_{i+1} - P_i)/\lVert P_{i+1} - P_i\rVert^2,\, 0,\, 1\big)$	Bus and segment endpoints (in flat x/y)	Fraction along segment + perp distance
On route?	$\min_i \delta_i \le W$	All segments' perp distances	True / False
ETA to stop P_m	Three-piece sum (current-seg leftover + later τ's + stop waits)	L_k, s_k, τ, d	Seconds
Learning update	$\tau \leftarrow \alpha T_{\text{obs}} + (1-\alpha)\tau$	Observed segment time	New running average

Job	Formula	What you plug in	What comes out
Dead-reckon speed	$\hat{v} = \gamma v_{\text{last}} + (1-\gamma) v_{\text{hist}}$, $\gamma = e^{-\Delta t/\kappa}$	Last speed, historical speed, blackout duration	Best-guess current speed

Six equations. Every one explained, every one with numbers you can recompute on a napkin. That's the whole math from the server side.

References & sources

Where this document leans on external material, here is the source by name. Verify URLs / exact author lists / document IDs before publishing — they are *named* below, not vouched for to bibliographic precision.

Math & methods - Haversine formula and Earth-radius value (R ≈ 6 371 000 m) — standard WGS-84; any geodesy text. - Ed Williams, "Aviation Formulary" — public reference for cross-track / point-to-segment formulas (§2–3). - Hyndman & Athanasopoulos, *Forecasting: Principles and Practice* (online textbook, otexts.com/fpp3) — exponential smoothing chapter, basis for §5. - arXiv:1904.05037 — travel-time prediction survey naming exponential smoothing as a mainstream baseline (§5). - *Transport* (Vilnius Tech), 2019 — paper integrating exponential smoothing with state-space modelling for bus arrival prediction (§5). - RFC 6298 (IETF) and Karn's algorithm — TCP RTT estimation, the analogy for EWMA in §5.

Hardware - TI MSPM0G3507 datasheet + MSPM0 SDK (ti.com) — source of all flash-bank, sector-size, endurance, write-granularity numbers in §6.6–6.10. - Winbond W25Q32JV SPI flash datasheet — backup-buffer option in §6.8. - SIMCom A7670 / SIM7600 module datasheets — 4G + 2G fallback modem. - u-blox NEO-6M GPS receiver datasheet.

Routing - OSRM (project-osrm.org) + OpenStreetMap — road geometry and seed travel-times (§8.3, simulator).

Connectivity / coverage (§6.8) - TRAI Performance Indicator Reports — rural-circle network availability, the principled source for blackout-duration bounds. The "~6 h worst-realistic" figure is a bounded estimate *from* this kind of data, not a measurement — explicitly flagged in §6.8. - TRAI MyCall + OpenSignal + nPerf — crowd-sourced signal-quality maps (alternative source).

SMS gateway providers (§6, Tier 2) - MSG91, Exotel, Twilio India pricing pages — for SMS cost (₹0.15–0.25/SMS). - TRAI DLT (sender-registration) regulations — required for transactional SMS in India.

Domain context - Tamil Nadu State Transport Corporation (tnstc.in) — fleet count and operator structure. - WiSH 2026 `Problem_Statement.pdf` — the 22 800-buses / 38-district figures originate here.